

Project Demonstration Phase

This phase provides a structured walk-through, script, and visual outline for presenting and demonstrating the **Implement Client Script & UI Policy (Incident)** solution to stakeholders, evaluators, or project teams.

1. Demonstration Agenda & Overview

DEMONSTRATION WALK-THROUGH SEQUENCE	
1. Introduction & Business Problem Overview	
2. Scene 1: High Impact Automation & Read-Only Policy (onChange + UI Policy)	
3. Scene 2: Submission Guardrails & Mandatory Validation (onSubmit Script)	
4. Scene 3: Reversibility & Dynamic UI Reset (UI Policy Reverse Engine)	
5. Scene 4: Governance & List View Restriction (onCellEdit Script)	
6. Scene 5: Form-Based State Update Verification	
7. Technical Summary & Q&A	

2. Demonstration Script & Execution Steps

Scene 1: High Impact Automation & Read-Only Policy

- **Objective:** Demonstrate automatic field population and policy-driven field locking upon setting Impact to High.
- **Action:**
 1. Navigate to **Incident** → **Create New** in ServiceNow.
 2. Select **1 – High** in the **Impact** drop-down field.
- **Talking Point:** *"Notice how selecting High Impact triggers two client-side events instantly. First, our onChange client script sets the Urgency field to High and throws an informational message banner. Second, our UI Policy kicks in, rendering the Urgency field read-only so users cannot manually modify it under high-impact conditions."*
- **Expected Result:**
 - Info message displayed: *"Urgency set to High for High impact incident."*
 - Urgency set to **1 - High** and locked (read-only).

Scene 2: Submission Guardrails & Mandatory Validation

- **Objective:** Showcase how onSubmit validation prevents incomplete incident creation.
- **Action:**
 1. Leave the **Assigned To** field completely blank.
 2. Click **Submit** or **Save** at the top of the form.
- **Talking Point:** *"Even if a user bypasses initial form fields, our onSubmit client script acts as a final gateway. Since this is a High Impact incident and Assigned To is empty, the form submission is aborted, and an inline field error explicitly directs the user to populate the assignee."*
- **Expected Result:**
 - Form does **not** submit.
 - Red inline error box highlights **Assigned To**: *"Assigned To is mandatory for High impact incidents."*

Scene 3: Reversibility & Dynamic UI Reset

- **Objective:** Demonstrate policy rollback when trigger conditions change.
- **Action:**
 1. Populate **Assigned To** with any valid user to satisfy validation.
 2. Change **Impact** from **1 – High** to **2 – Medium**.
- **Talking Point:** *"Because we configured 'Reverse if false' on our UI Policy, changing Impact away from High immediately unlocks the Urgency field without needing a page refresh or complex client-side reset logic."*
- **Expected Result:**
 - Urgency field becomes editable again.
 - Incident record can be saved cleanly.

Scene 4: Governance & List View Restriction

- **Objective:** Show how inline list editing is blocked for critical status fields.
- **Action:**
 1. Navigate to **Incident** → **All** list view.
 2. Double-click the **State** column on any listed incident.
- **Talking Point:** *"To maintain process control, we use an onCellEdit script. If a user tries to modify the State field directly from the list view, the action is intercepted, an alert is displayed, and callback(false) rejects the cell edit."*
- **Expected Result:**
 - Alert window pops up: *"State cannot be updated using list editing. Please open the Incident."*
 - State value remains unchanged in the list view.

Scene 5: Form-Based State Update Verification

- **Objective:** Confirm that authorized form-based updates still function seamlessly.
- **Action:**
 1. Click to open an Incident record.
 2. Change **State** on the Incident form and click **Update**.
- **Talking Point:** *"The list-edit block only applies to quick list edits. Standard form-based updates to State continue to function as expected, ensuring proper workflows are"*

followed."

- **Expected Result:**
 - Incident form updates and saves successfully.

3. Key Demonstration Takeaways

- **Enhanced Data Integrity:** High-impact incidents are forced to have valid assignment and priority metrics before record creation.
- **Zero Technical Debt Architecture:** UI Policies handle field UI states (Read-only), while lightweight Client Scripts handle explicit event actions (onChange, onSubmit, onCellEdit).
- **User Experience & Governance:** Clear warning messages prevent user error, and list-edit controls enforce proper form-level ticket progression.